iSAQB Advanced DSL - Lambda Calculus

Michael Sperber

Created: 2026-04-10 Fri 10:52

Recurring Problems in Language Design

- naming
- semantics
- implementation
- "essence"

(LG 3-3) The Next 700 Programming Languages

A Correspondence Between ALGOL 60 and Church's Lambda-Notation: Part I*

By P. J. Landin†

This paper describes how some of the semantics of ALGOL 60 can be formalized by establishing a correspondence between expressions of ALGOL 60 and expressions in a modified form of Church's λ-notation. First a model for computer languages and computer behavior is described, based on the notions of functional application and functional abstraction, but also having analogues for imperative language features. Then this model is used as an "abstract object language" into which ALGOL 60 is mapped. Many of ALGOL 60's features emerge as particular arrangements of a small number of structural rules, suggesting new classifications and generalizations.

The correspondence is first described informally, mainly by illustrations. The second part of the paper gives a formal description, i.e. an "abstract compiler" into the "abstract object language." This is itself presented in a "purely functional" notation, that is one using only application and abstraction.

Introduction

Anyone familiar with both Church's λ-calculi (see e.g. [7]) and Algol 60 [6] will have noticed a superficial resemblance between the way variables tie up with the λ's in a nest of λ-expressions, and the way identifiers tie up with the headings in a nest of procedures and blocks. Some may also have observed that in, say

$$\{\lambda f.f(a) + f(b)\}[\lambda x.x^2 + px + q]$$

the two λ-expressions, i.e. the *operator* and the *operand*, play roughly the roles of block-body and procedure-declaration, respectively. The present paper explores this resemblance in some detail.

The presentation falls into four sections. The first section, following this introduction, gives some motivation for examining the correspondence. The second section describes an abstract language based on Church's λ-calculi. This abstract language is a development of the AE/SECD system presented in [3] and some acquaintance with that paper (hereinafter referred to as [MEE]) is assumed here. The third section describes informally, mainly by illustrations, a correspondence between expressions of Algol 60 and expressions of the abstract language. The last section formalizes this correspondence; it first describes a sort of "abstract Algol 60" and then presents two func-

Landin: Correspondence between ALGOL 60 and Church's Lambda-notation: part I (1965)

(LG 3-2) Lambda Calculus - Language

$$
\begin{aligned}
\langle \mathcal{L}_\lambda \rangle \rightarrow \quad & \langle V \rangle \\
| \quad & (\langle \mathcal{L}_\lambda \rangle \, \langle \mathcal{L}_\lambda \rangle) \\
| \quad & (\lambda \, \langle V \rangle . \, \langle \mathcal{L}_\lambda \rangle)
\end{aligned}
$$

(Variable, Application, Abstraction)

Lambda Calculus - Examples

x	Variable
$(f\ x)$	Application
$((f\ x)\ (\lambda x.\ x))$	Application
$(\lambda x.\ (\lambda y.\ (x\ y))$	Abstraction
$(\lambda x.\ (f\ x))$	Abstraction
$(\lambda x.\ (f\ (\lambda x\ x))$	Abstraction
$((\lambda x.\ x)\ (\lambda x.\ x))$	Application

Abbreviations

abbreviation	stands for
$\lambda \mathrm{f}.\, \lambda \mathrm{x}.\, \mathrm{f}\ \mathrm{x}$	$(\lambda \mathrm{f}.\, (\lambda \mathrm{x}.\, (\mathrm{f}\ \mathrm{x})))$
$\lambda x_1 \ldots x_n.\, e$	$\lambda x_1.\, (\lambda x_2.\, (\ldots \lambda x_n.\, e) \ldots)$
$e_0 \ldots e_n$	$(\ldots (e_0\ e_1)\ e_2) \ldots e_n)$

"Body of a λ extends to the right as far as possible."

$$
\begin{aligned}
(\mathrm{f}\ \mathrm{x}) &\quad \mathrm{f}\ \mathrm{x} \\
(\mathrm{f}\ \mathrm{x})\ \mathrm{y} &\quad \mathrm{f}\ \mathrm{x}\ \mathrm{y} \\
(\lambda \mathrm{x}.\, (\mathrm{x}\ \ x)) &\quad \lambda \mathrm{x}.\, \mathrm{x}\ \mathrm{x} \\
(\lambda \mathrm{x}.\, ((\mathrm{x}\ \mathrm{y})\ \mathrm{z})) &\quad \lambda \mathrm{x}.\, \mathrm{x}\ \mathrm{y}\ \mathrm{z} \\
(\lambda \mathrm{x}.\, (\lambda\ \mathrm{y}.\, (\mathrm{x}\ \mathrm{y}))) &\quad \lambda \mathrm{xy}.\, \mathrm{x}\ \mathrm{y}
\end{aligned}
$$

Exercise

Expand the following terms:

$$\mathrm{f\ x\ y\ z}$$

$$\lambda \mathrm{xyz.\ x\ y\ z}$$

$$((\lambda \mathrm{xy.\ x\ y})\ \mathrm{a\ b})$$

Free and Bound Variables

$$\text{free}(e) := \begin{cases} \{v\} & \text{falls } e = v \\ \text{free}(e_0) \cup \text{free}(e_1) & \text{if } e = e_0\ e_1 \\ \text{free}(e_0) \setminus \{v\} & \text{if } e = \lambda v.\, e_0 \end{cases}$$

$$\text{bound}(e) := \begin{cases} \varnothing & \text{if } e = v \\ \text{bound}(e_0) \cup \text{bound}(e_1) & \text{if } e = e_0\ e_1 \\ \text{bound}(e_0) \cup \{v\} & \text{if } e = \lambda v.\, e_0 \end{cases}$$

Exercise

$$\mathrm{free}(\mathrm{x\ y})$$
$$\mathrm{free}(\lambda \mathrm{x}.\ \mathrm{x\ y})$$
$$\mathrm{free}(\lambda \mathrm{xy}.\ \mathrm{x\ y})$$
$$\mathrm{free}((\lambda \mathrm{x}.\ \mathrm{y})\ \mathrm{x})$$
$$\mathrm{bound}(\mathrm{x\ y})$$
$$\mathrm{bound}(\lambda \mathrm{x}.\ \mathrm{x\ y})$$
$$\mathrm{bound}(\lambda \mathrm{xy}.\ \mathrm{x\ y})$$
$$\mathrm{bound}((\lambda \mathrm{x}.\ \mathrm{y})\ \mathrm{x})$$

Substitution

For $e, f \in \mathcal{L}_\lambda$:

$$e[x \mapsto f] := \begin{cases} f & \text{if } e = x \\ e & \text{if } e \text{ is a variable and } e \neq x \\ \lambda x.\, e_0 & \text{if } e = \lambda x.\, e_0 \\ \lambda y.\, (e_0[x \mapsto f]) & \text{if } e = \lambda y.\, e_0 \\ & \text{and } y \neq x, y \notin \mathrm{free}(f) \\ \lambda y'.\, (e_0[y \mapsto y'][x \mapsto f]) & \text{if } e = \lambda y.\, e_0 \\ & \text{and } y \neq x, y \in \mathrm{free}(f) \\ & \text{and } y' \notin \mathrm{free}(e_0) \cup \mathrm{free}(f) \\ (e_0[x \mapsto f])\ (e_1[x \mapsto f]) & \text{if } e = e_0\ e_1 \end{cases}$$

Substitution Example

$$(\lambda x.\, \lambda y.\, x\ (\lambda z.\, z)\ z)[z \mapsto x\ y]$$

$$\begin{aligned}
&= \lambda x'.\, ((\lambda y.\, x\ (\lambda z.\, z)\ z)[x \mapsto x'][z \mapsto x\ y]) \\
&= \lambda x'.\, ((\lambda y.\, ((x\ (\lambda z.\, z)\ z)[x \mapsto x']))[z \mapsto x\ y]) \\
&= \lambda x'.\, ((\lambda y.\, (x[x \mapsto x']\ ((\lambda z.\, z)[x \mapsto x'])\ z[x \mapsto x']))[z \mapsto x\ y]) \\
&= \lambda x'.\, ((\lambda y.\, (x'\ (\lambda z.\, z)\ z))[z \mapsto x\ y]) \\
&= \lambda x'.\, \lambda y'.\, ((x'\ (\lambda z.\, z)\ z)[y \mapsto y'][z \mapsto x\ y]) \\
&= \lambda x'.\, \lambda y'.\, ((x'[y \mapsto y']\ ((\lambda z.\, z)[y \mapsto y'])\ z[y \mapsto y'])[z \mapsto x\ y]) \\
&= \lambda x'.\, \lambda y'.\, ((x'\ (\lambda z.\, z)\ z)[z \mapsto x\ y]) \\
&= \lambda x'.\, \lambda y'.\, x'[z \mapsto x\ y]\ ((\lambda z.\, z)[z \mapsto x\ y])\ z[z \mapsto x\ y] \\
&= \lambda x'.\, \lambda y'.\, x'\ (\lambda z.\, z)\ (x\ y)
\end{aligned}$$

Reduction

$$\lambda x.\, e \to_\alpha \lambda y.\, (e[x \mapsto y]) \quad y \notin \mathrm{free}(e)$$
$$(\lambda x.\, e)\ f \to_\beta e[x \mapsto f]$$

A λ term has at most one normal form modulo α reduction.

Evaluation - Call-by-Name

- For shape $(\lambda x.\, e)\ f$, the whole term is the redex.
- For shape $e\ f$, and if e is **not** an abstraction, find the redex within e - on the **left**.
- For shape $e\ f$, and if e is **not** an abstraction and when there's no redex in e, find the redex within f.

Evaluation - Call-by-Value

- For shape $e\ f$, find the redex within e - **leftmost-innermost**.
- For shape $e\ f$, and if there's no redex in e, find redex within f.
- For shape $(\lambda x.\, e)\ f$, and if there's no redex in f, the whole term is the redex.

(LG 3-2) Exercise: Expressiveness of the lambda calculus

- How could you represent binary conditionals in the lambda calculus?

(LG 3-2) Fixpoints

$$Y = \lambda \mathrm{f}.\,(\lambda \mathrm{x}.\,\mathrm{f}\ (\mathrm{x}\ \mathrm{x}))\ (\lambda \mathrm{x}.\,\mathrm{f}\ (\mathrm{x}\ \mathrm{x}))$$

$$\forall F \in \mathcal{L}_\lambda : F(Y\ F) \equiv Y\ F$$

(LG 3-2) Church-Turing Thesis

All equivalent:

- general recursive functions (Gödel)
- Lambda calculus
- Turing machines

Applied Lambda Calculus

$B := \{\texttt{\#f}, \texttt{\#t}, 0, 1, 2, \ldots\}$ is the set of **base values**.

For n a natural number, Σ^n is the set of n-ary **primitives**

For each $F \in \Sigma^n$ there is an n-ary function F^{op} - its **operation**.

$V := B \cup \{(\lambda x.\, e) \mid x \text{ variable}, e \text{ term}\}$ is the set of **values**.

$b \in B, v \in V$ implicitly

Applied Lambda Calculus

$$
\begin{aligned}
\langle \mathcal{L}_{\lambda A} \rangle \rightarrow \quad & \langle V \rangle \\
| \quad & (\langle \mathcal{L}_{\lambda A} \rangle \, \langle \mathcal{L}_{\lambda A} \rangle) \\
| \quad & (\lambda \, \langle V \rangle . \, \langle \mathcal{L}_{\lambda A} \rangle) \\
| \quad & \langle B \rangle \\
| \quad & (\langle \Sigma^1 \rangle \; \langle \mathcal{L}_{\lambda A} \rangle) \\
| \quad & (\langle \Sigma^2 \rangle \; \langle \mathcal{L}_{\lambda A} \rangle \; \langle \mathcal{L}_{\lambda A} \rangle) \\
\ldots & \\
| \quad & (\langle \Sigma^n \rangle \; \langle \mathcal{L}_{\lambda A} \rangle \; \ldots \; \langle \mathcal{L}_{\lambda A} \rangle) \quad (n \text{ times})
\end{aligned}
$$

Applied Lambda Calculus

$$(F\ b_1\ \ldots\ b_n) \rightarrow_\delta F^{\mathrm{op}}(b_1, \ldots, b_n)$$

(LG 1-2, LG 3-3) Call-by-Value Operational Semantics

$$\frac{(\lambda x.\, e)\ v \to_\beta e[x \mapsto f]}{(\lambda x.\, e)\ v \to_{\mathrm{cbv}} e[x \mapsto f]}$$

$$\frac{e_1 \to_{\mathrm{cbv}} e_1'}{e_1\ e_2 \to_{\mathrm{cbv}} e_1'\ e_2}$$

$$\frac{e_2 \to_{\mathrm{cbv}} e_2'}{v\ e_2 \to_{\mathrm{cbv}} v\ e_2'}$$

Call-by-Value Operational Semantics

$$\frac{F \in \Sigma^n \qquad F\ b_1\ \ldots\ b_n \rightarrow_\delta F^{\mathrm{op}}(b_1, \ldots, b_n)}{F\ v_1\ \ldots\ b_n \rightarrow_{\mathrm{cbv}} F^{\mathrm{op}}(b_1, \ldots, b_n)}$$

$$\frac{F \in \Sigma^n \qquad e_i \rightarrow_{\mathrm{cbv}} e_i'}{F\ b_1 \ldots b_{i-1}\ e_i\ e_{i+1}\ e_n \ldots \rightarrow_{\mathrm{cbv}} F\ b_1 \ldots b_{i-1}\ e_i'\ e_{i+1} \ldots e_n}$$

(LG 3-1) Call-by-Value "Big Step" Evaluation Semantics

$$\Gamma \vdash \lambda x.\, e \Downarrow \lambda x.\, e$$

$$\Gamma \vdash v \Downarrow v$$

$$\frac{\Gamma(x) = v}{\Gamma \vdash x \Downarrow v}$$

$$\frac{\Gamma \vdash e_1 \Downarrow \lambda x.\, e \qquad \Gamma \vdash e_2 \Downarrow v_2 \qquad \Gamma[x \mapsto v_2] \vdash e \Downarrow v}{\Gamma \vdash e_1\, e_2 \Downarrow v}$$

Call-by-Value Reduction Semantics

$$
\begin{array}{rll}
C \rightarrow & [\cdot] & \textit{evaluation context} \\
\mid & C\ \langle \mathcal{L}_{\lambda A} \rangle & \\
\mid & \langle V \rangle\ C & \\
\mid & \langle \Sigma^n \rangle\ \langle B \rangle\ \ldots\ \langle B \rangle\ C\ \langle \mathcal{L}_{\lambda A} \rangle\ \ldots\ \langle \mathcal{L}_{\lambda A} \rangle &
\end{array}
$$

$$
\begin{array}{lcl}
C[(\lambda x.\, e)\ f] & \rightarrow_{\mathrm{cbv}} & C[e[x \mapsto f]] \\
C[F\ b_1\ \ldots\ b_n] & \rightarrow_{\mathrm{cbv}} & C[F^{\mathrm{op}}(b_1, \ldots, b_n)]
\end{array}
$$

(LG 3-3) Denotational Semantics

- Function from expressions to mathematical object (*denotation*).
- Typically *continuous* function between *domains* (*complete partial orders*).

(LG 3-3) Denotational Semantics of the Lambda Calculus

$$D_\infty \quad \underset{\psi}{\overset{\phi}{\rightleftarrows}} \quad [D_\infty \to D_\infty]$$

$$\begin{aligned}
[\![-]\!] &\in \mathcal{L}_\lambda \to [Env \to D_\infty] \\
[\![x]\!]\eta &= \eta(v) \\
[\![\lambda x.\, e]\!]\eta &= \psi(\lambda v \in D_\infty.\, [\![e]\!]\eta[x \mapsto v]) \\
[\![e_1\ e_2]\!]\eta &= \phi([\![e_1]\!]\eta)\ ([\![e_2]\!]\eta)
\end{aligned}$$

(LG 3-3) Exercise: Informal Semantics of Tim

Describe the semantics of Tim using appropriate notation and language!

(LG 5-2) PLT Redex: Lambda Calculus

```
(define-language Lambda
  (e ::=
     x
     (lambda (x_!_ ...) e)
     (e e ...))
  (x ::= variable-not-otherwise-mentioned))
```

PLT Redex: Evaluation Context

```
(define-extended-language Lambda-calculus Lambda
  (e ::= .... n)
  (v ::= n (lambda (x ...) e))
  (n ::= number)
  (C ::= hole (e ... C e ...) (lambda (x_!_ ...) C)))
```

PLT Redex: Reduction Relation

```
(define -->βv
  (reduction-relation
   Lambda-calculus
   (--> (in-hole C ((lambda (x_1 ..._n) e) v_1 ..._n))
        (in-hole C (subst ([v_1 x_1] ...) e))
        βv)))
```